

Master Team Project Summer 2025

UniMarkt

WWW site for buy/sell/exchange exclusively for Fulda
students, staff and faculty

Team 2

Nirajan Khatri (NK)
nirajan.khatri@informatik.hs-fulda.de
Team Lead

Murtaza Muffazal Dhariwala (MMD)
murtaza-muffazal.dhariwala@informatik.hs-fulda.de
Frontend Lead

Hasara Nimashi Koralege (HNK)
hasara-nimashi.koralege@informatik.hs-fulda.de
Backend Lead

Rohan Kolagada (RK)
rohan.kolagada@informatik.hs-fulda.de
GitHub Master

Inchara Rao Konagolli Shrikanth (IRKS)
inchara-rao.konagolli-shrikanth@informatik.hs-fulda.de
Backend Developer

Muhammad Azeem Taufiq (MAT)
muhammad-azeem.taufiq@informatik.hs-fulda.de
DataBase Developer

Milestone 2

Submission on .05.2025

Prof. Dr. Rainer Todtenhöfer
Class CEO/CTO

<u>Revision</u>	<u>Revision Date</u>	<u>Submission Date</u>	<u>Author(s)</u>	<u>Description</u>
2.0 (Milestone 2)			NK, MMD, HNK, RK, IRKS, MAT	First version of Milestone 2, revised by team only

Table of Contents

1. Functional Requirements - prioritized	2
2. List of main data items and entities	8
3. UI Mockups and Storyboards	9
4. High level Architecture, Database Organization	9
5. High Level UML Diagrams	10
6. Identify actual key risks for your project at this time	13
7. Project management	16

1. Functional Requirements - prioritized

Priority 1: Must-Have Requirements

Unregistered Users

1. View Item Listings

- Allow unregistered users to browse all approved items for sale, services, and jobs, displaying a summary view with title, thumbnail image, location, price, and category.
- Listings are paginated and include a brief description (truncated to 100 characters). Users can click to view full details but cannot interact further without logging in.

2. User Registration

- Allow new users to register using an email address ending with “@hs-fulda.de”, providing a password.
- 2.1 Validate email domain to ensure it ends with “@hs-fulda.de”.
- 2.2 Ensure password meets strength criteria (min 8 characters, includes letters and numbers).
- 2.3 Store user data securely.
- 2.4 Prevent login until email is verified.

Registered Users

3. User Login and Logout

- Enable registered users to log in using their “@hs-fulda.de” email and password, and log out securely.
- 3.1 Implement session management with a timeout.
- 3.2 Secure logout functionality redirects to the homepage.

4. Post an Item for Sale

- Enable users to create a listing with item title, description, category, images (up to 5), location, and asking price.
- 4.1 Validate title (max 100 characters).
- 4.2 Validate description (max 1000 characters).
- 4.3 Ensure category is selected from admin-defined options.
- 4.4 Validate price (positive number).
- 4.5 Validate location.
- 4.6 Validate images (JPEG/PNG, max 5MB each, up to 5 images).

5. Edit Own Item Listings

- Enable users to modify their active item listings.

- 5.1 Edit listing fields (title, description, category, images, price, location).
- 5.2 Set edited listing status to "pending" for admin re-approval.
- 6. Delete Own Item Listings
 - Enable users to delete their active item listings.
 - 6.1 Mark listing as "deleted" in the database.
 - 6.2 Remove from public view but retain for admin audit.
- 7. Search Listings
 - Allow users to search items by keyword, category, location, or price range.
 - 7.1 Search by keyword using SQL %LIKE for title and description.
 - 7.2 Search by category.
 - 7.3 Search by location.
 - 7.4 Search by price range.
- 8. Filter and Sort Listings
 - Enable users to filter listings by category or condition (e.g., new, used) and sort by date (newest/oldest) or price (low/high).
 - 8.1 Filter by category.
 - 8.2 Sort by date.
 - 8.3 Sort by price.
- 9. Item Detail Page
 - Show detailed information about a selected item, including images, description, seller name (or pseudonym), location, and price.
 - 9.1 Display all uploaded images in a carousel (one hero image and up to 4 others).
 - 9.2 Show full description, category, price, and location.
 - 9.3 Include a "Message Seller" button linking to the messaging system.
- 10. In-Site Messaging
 - Provide a secure messaging system for buyers and sellers to communicate within the platform.
 - 10.1 Store messages in the database tied to specific listings.
 - 10.2 Provide a user inbox with notifications (e.g., on-site or email) for new messages.
- 11. Mark Items as Sold
 - Allow sellers to update the status of their item to “sold” once the transaction is complete.
 - 11.1 Update listing status.
 - 11.2 Remove from public view.

- 11.3 Retain the user's history.

Admins

12. Admin Item Approval

- Allow an admin to review and approve/reject items before they are visible to users.
- 12.1 Access a pending listings dashboard.
- 12.2 Approve listings meeting standards.
- 12.3 Reject listings with a reason.

13. Category Management

- Allow admins to create, edit, or remove item categories.
- 13.1 Create new categories (e.g., “Electronics”).
- 13.2 Edit existing category names.

14. View User List

- Enable admins to view a list of all registered users on the platform.
- 14.1 Display a paginated user list.
- 14.2 Show user details (username, email, registration date, status).
- 14.3 Search users by name or email.

15. View Job List

- Enable admins to view a list of all job postings on the platform.
- 15.1 Display a paginated job list.
- 15.2 Show job details (title, professor, department, date, status).
- 15.3 Search jobs by title.
- 15.4 Sort jobs by date.

16. View Product List

- Enable admins to view a list of all product (item) listings on the platform.
- 17.1 Display a paginated product list.
- 17.2 Show product details (title, seller, category, price, status).
- 17.3 Search products by title or seller.
- 17.4 Sort products by date or status.
- 17.5 Filter products by category or price range.

17. Approve Products

- Allow admins to approve product listings submitted by users, making them visible to the public.
- 18.1 Review product details (title, description, images, category, price).
- 18.2 Approve products meeting platform standards.

- 18.3 Make approved products immediately visible in public listings.

18. Reject Products

- Allow admins to reject product listings that do not meet platform standards, providing feedback to the user.
- 20.1 Select a rejection reason (e.g., "inappropriate content").

19. Delete Users

- Enable admins to permanently remove user accounts that violate site policies or are inactive.
- 23.1 Mark user account as "deleted" in the database.
- 23.2 Anonymize or delete associated data.

20. Delete Products

- Enable admins to permanently remove product listings that violate site policies or are no longer needed.
- 24.1 Mark product as "deleted" in the database.
- 24.2 Retain for audit purposes.

Priority 2: Desired Requirements

Faculty

25. Create a Job Posting

- Enable faculty to post job opportunities, such as research or tutoring positions.
- 25.1 Provide job title.
- 25.2 Provide job description.
- 25.3 Provide contact information.
- 25.4 Provide department.
- 25.5 Provide remuneration.

26. Edit Job Posting

- Enable faculty to modify their active job postings.
- 26.1 Edit job fields.
- 26.2 Set edited posting status to "pending" for admin re-approval.

27. Delete Job Posting

- Enable faculty to delete their active job postings.
- 27.1 Mark job as "deleted" in the database.
- 27.2 Remove from public view but retain for admin audit.

28. Archive a Job Posting

- Allow faculty to archive their job postings, making them invisible to other users.
- 28.1 Provide an "Archive" option for each active job posting.
- 28.2 When archived, the job is no longer visible in public listings but remains accessible to the faculty.
- 28.3 Update the job status to "archived" in the database.

Registered Users

30. View Job Postings

- Allow users to explore available job postings.
- 30.1 Browse active postings.
- 30.2 Filter by department.
- 30.3 Search by keywords.

31. Apply for a Job

- Enable users to submit applications for job postings.
- 31.1 Upload resume (PDF, max 5MB).
- 31.2 Upload cover letter (text or PDF).

32. List a Service

- Enable users to offer services like training, study help, or photography.
- 32.1 Input service title.
- 32.2 Input service description.
- 32.3 Select category (e.g., tutoring, photography).
- 32.4 Set pricing (hourly or fixed).
- 32.5 Specify description.
- 32.6 Listings are "pending" until admin approval.

33. Edit Service Listing

- Enable users to modify their active service listings.
- 33.1 Edit service fields.
- 33.2 Set edited service status to "pending" for admin re-approval.

34. Delete Service Listing

- Enable users to delete their active service listings.
- 34.1 Mark service as "deleted" in the database.
- 34.2 Remove from public view but retain for admin audit.

35. View Available Services

- Allow users to browse and search for offered services.
- 35.1 Browse approved services.
- 35.2 Filter by category and price.

- 35.3 Search by keywords.

36. Archive a Service

- Allow users to archive their service listings, making them invisible to other users.
- 36.1 Provide an "Archive" option for each active service listing.
- 36.2 When archived, the service is no longer visible in public listings but remains accessible to the user.
- 36.3 Update the service status to "archived" in the database.

37. Unarchive a Service

- Allow users to unarchive their archived service listings, restoring them to public view.
- 37.1 Provide an "Unarchive" option for each archived service listing.
- 37.2 When unarchived, the service becomes visible again in public listings.
- 37.3 Update the service status to "active" in the database.

38. Create an Auction Listing

- Enable registered users to create auction listings for items they wish to sell through a bidding process.
- 38.1 Allow users to input an item title (max 100 characters), description (max 1000 characters), category (selected from admin-defined options), starting bid (positive number), reserve price (optional, positive number), auction duration (1–7 days), and up to 5 images (JPEG/PNG, max 5MB each).
- 38.2 Set the listing status to “pending” until admin approval.
- 38.3 Notify the user via when the listing is submitted for approval.

39. Bid on an Auction

- Enable registered users to place bids on active auction listings.
- 42.1 Ensure each bid is higher than the current highest bid or the starting bid.
- 42.2 Validate that only logged-in users can place bids.
- 42.3 Notify the bidder upon successful bid placement..

40. View Auction Details

- Display detailed information about a selected auction listing.
- 43.1 Show all uploaded images in a carousel (one hero image and up to 4 others), full description, category, current highest bid, number of bids, time remaining, and seller name (or pseudonym).
- 43.2 Include a “Place Bid” button for logged-in users, linking to the bidding interface.
- 43.3 Display a countdown timer showing the remaining time until the auction ends.
- 43.4 Allow unregistered users to view auction details but disable the “Place Bid” button.

41. Mark Auction as Completed

- Allow the seller to mark an auction as completed once the transaction with the highest bidder is finalized.
- 45.1 Update the auction status to “completed” in the database.
- 45.2 Remove the auction from public view but retain it in the seller’s history and database for audit purposes.
- 45.3 Notify the highest bidder and seller when the auction is marked as completed.

Admins

42. Approve a Service

- Allow admins to approve service listings submitted by users, making them visible to the public.
- 36.1 Review service details (title, description, category, price).
- 36.2 Approve services meeting platform standards.
- 36.3 Make approved services immediately visible in public listings.

43. Unarchive a Service

- Allow users to unarchive their archived service listings, restoring them to public view.
- 37.1 Provide an "Unarchive" option for each archived service listing.
- 37.2 When unarchived, the service becomes visible again in public listings.
- 37.3 Update the service status to "active" in the database.

Admins

44. Approve Auction Listings

- Allow admins to review and approve auction listings before they are visible to users.
- 41.1 Provide access to a pending auctions dashboard displaying title, description, images, category, starting bid, reserve price (if set), and duration.
- 41.2 Enable admins to approve listings that meet platform standards, making them immediately visible in public listings.
- 41.3 Allow admins to reject listings with a predefined reason (e.g., “inappropriate content,” “missing information”) and optional comments, notifying the seller via in-site notification.

45. Reject Auction Listings

- Allow admins to reject auction listings that do not meet platform standards, providing feedback to the user.
- 48.1 Require selection of a rejection reason from predefined options.
- 48.2 Allow optional comments for additional feedback.

- 48.3 Notify the seller via in-site notification with the rejection reason and instructions to edit and resubmit.
- 48.4 Store rejection reasons in the database for audit purposes.

46. Delete Auction Listings

- Enable admins to permanently remove auction listings that violate site policies or are no longer needed.
- 49.1 Mark the auction as “deleted” in the database but retain it for audit purposes.
- 49.2 Remove the auction from public view and user dashboards.
- 49.3 Notify the seller and any bidders via in-site notification with the reason for deletion.

Priority 3: Opportunistic Requirements

Registered Users

47. Save Favorites / Wishlist

- Allow users to save listings to a favorites list for quick access later.
- 50.1 Add/remove listings to wishlist via a button on the listing or detail page.
- 50.2 Access wishlist from the user’s profile.
- 50.3 Display summary information (e.g., title, price, category). Listings remain saved unless removed or marked as sold.

48. AI-Powered Item and Service Recommendations

- Provide personalized recommendations for items, services, and jobs based on user activity and preferences.
- 51.1 Analyze user data (e.g., search history, viewed listings, past purchases, or bids) to generate recommendations using a machine learning model.
- 51.2 Display recommendations on the homepage, user dashboard, or item/service detail pages, showing up to 5 relevant listings (e.g., “You might like” section) with title, thumbnail, price, and category.
- 51.3 Allow users to opt out of personalized recommendations via profile settings to comply with privacy preferences.
- 51.4 Periodically update recommendations based on new user interactions (e.g., weekly or after significant activity like a purchase or bid).

49. User Feedback and Rating System

- Enable registered users to leave feedback and ratings for completed transactions (e.g., item sales, service bookings, or auction wins).
- 52.1 Allow buyers to rate sellers (1–5 stars) and provide optional comments after marking a transaction as completed.

- 52.2 Display the seller's average rating and number of reviews on their profile and listing detail pages.
- 52.3 Allow sellers to respond to feedback publicly or privately via in-site messaging to address concerns.
- 52.4 Implement moderation by admins to review flagged or inappropriate feedback, with the ability to remove or edit comments.

2. List of main data items and entities

This section defines the core entities, user types, and key data structures of the UniMarkt platform at a high level. Privileges are included where applicable to support user role-driven behavior.

1. **User Entities:** John Doe, Fulda University-affiliated user (student, staff, faculty) is classified as a Buyer, Seller, Pro User, Job Hunter, Employer, Moderator, Skill Provider, or Skill Seeker. Attributes include `user_id`, `full_name`, university email, `user_roles`, `joined_date`, optional `profile_photo`, and `is_verified` status. Privileges: Buyers browse/contact sellers, Sellers manage listings, Job Hunters apply for jobs, Employers post jobs, Moderators manage content/users, Skill Providers offer services, Skill Seekers purchase services.
2. **Marketplace Entities:** An item, listed by a Seller (e.g., books, electronics), has an `item_id`, `title`, `description`, `category`, `price`, `condition`, `image_urls`, `posted_by`, `posted_date`, and `status`. It appears in a searchable, filterable feed after admin moderation.
3. **Message Entities:** A message, identified by `message_id`, includes `sender_id`, `receiver_id`, `context` (item/job/skill), `body`, and `timestamp`. Private to involved users, it supports negotiations, job applications, or skill arrangements.
4. **Skill Offering Entities:** A paid service (e.g., tutoring), identified by `skill_id`, includes `title`, `description`, `category`, `offered_by`, `price`, `availability_schedule`, and `status`. Skill Seekers can request services, with payments handled on/off-platform and optional reviews.
5. **Job Portal Entities:** Job ListingA student job (e.g., HiWi), identified by `job_id`, includes `title`, `department`, `description`, `qualifications`, `posted_by`, `contact_email`, `posted_date`, and `status`. Visible post-moderation, filterable by students.
6. **Category Entities:** A category (e.g., Books, Academic Support), identified by `category_id`, includes a name and optional icon, aiding organization and navigation of items and skills.

3. UI Mockups and Storyboards

<https://github.com/Rohan-Kolagada/GDSDSummer2025/blob/98c2f1eff5890db8fbfc96db70dfde9d230b304d/Milestones/Fulda%20SS2025%20UI%20Mockups%20Document%20Team%202.pdf>

4. High level Architecture, Database Organization

4.1 Database Organization

The system uses a relational MySQL database with the following main tables:

- Role: Defines user access levels (e.g., admin, student).
- User: Stores user credentials, security info, and role link.
- Profile: Additional user data like phone number and qualifications.
- Category/SubCategory: Hierarchical classification of product listings.
- Product: Marketplace items, includes description, price, images, location, etc.
- Department: Academic departments for skill/job alignment.
- Skill: Tutoring or service listings linked to users and departments.
- Job: Job listings with title, description, qualifications, and contact info.
- Message: User messaging system tracking communication threads.

4.2 Media Storage

- Images are stored externally using Amazon S3. Database fields hold only URLs or file paths.
- No BLOBs are stored in the DB to optimize scalability and performance.
- No special handling for video, audio, or GPS data is required.

4.3 Search & Filter Architecture

The system uses SQL queries for search and filtering.

Search Methods:

- ILIKE for case-insensitive matching
- %LIKE% for partial text searches

Filtering via WHERE, BETWEEN, and Boolean logic.

Searchable Fields:

- Items: Search in description; filter by category, price.
- Skills: Search by description/module; filter by department and price.
- Jobs: Search by title/description/qualifications; filter by department.

- Categories/Departments: Filter using linked IDs for hierarchical browsing.

Frontend organizes results by type into tabs: Items, Tutoring, Jobs. Each includes custom filter inputs.

4.4 Custom APIs

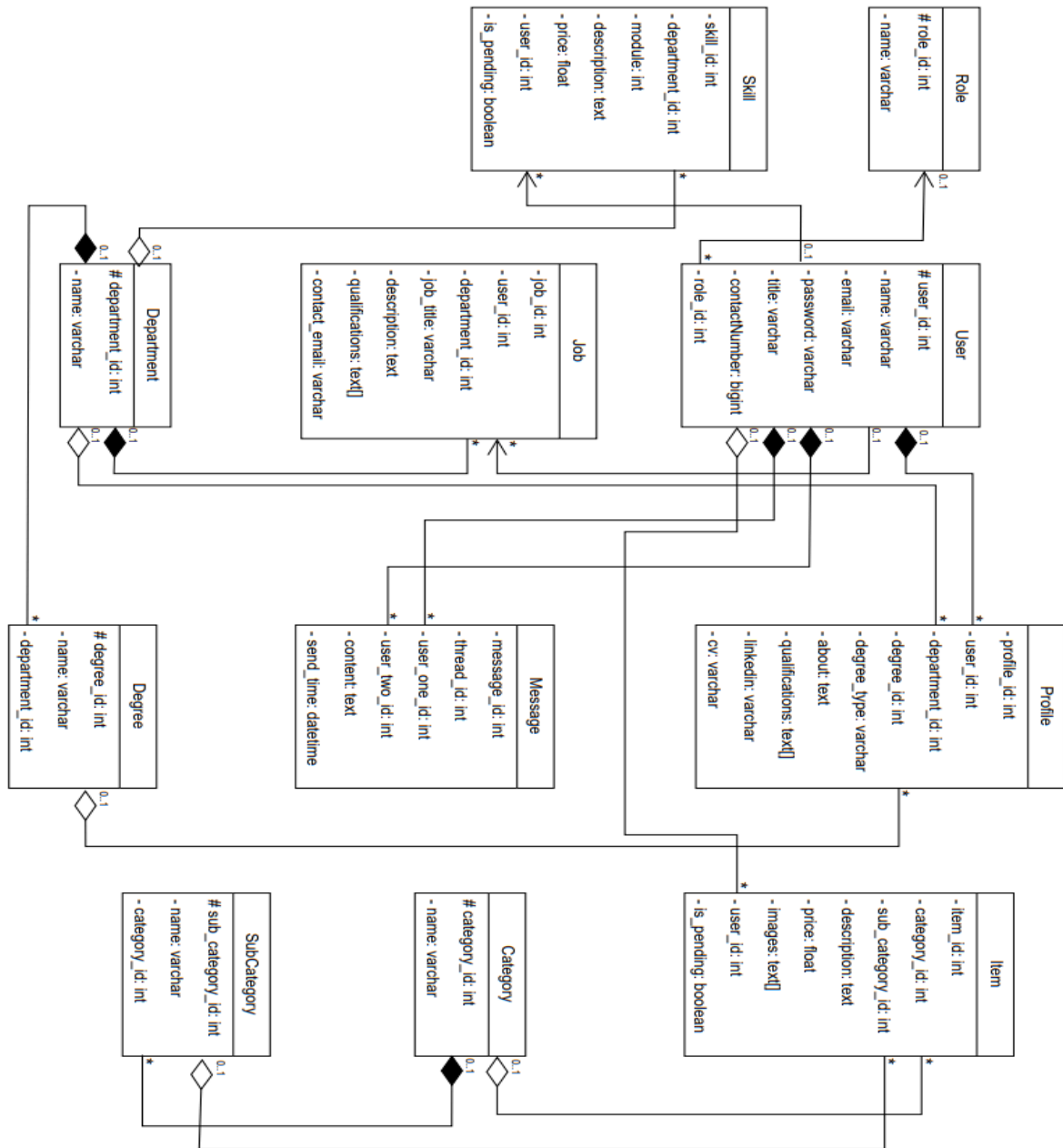
Custom RESTful APIs are implemented to support:

- Item, skill, and job search (e.g., /items/search, /skills/search, /jobs/search)
- User profile management and authentication
- Messaging between users (threaded conversations)

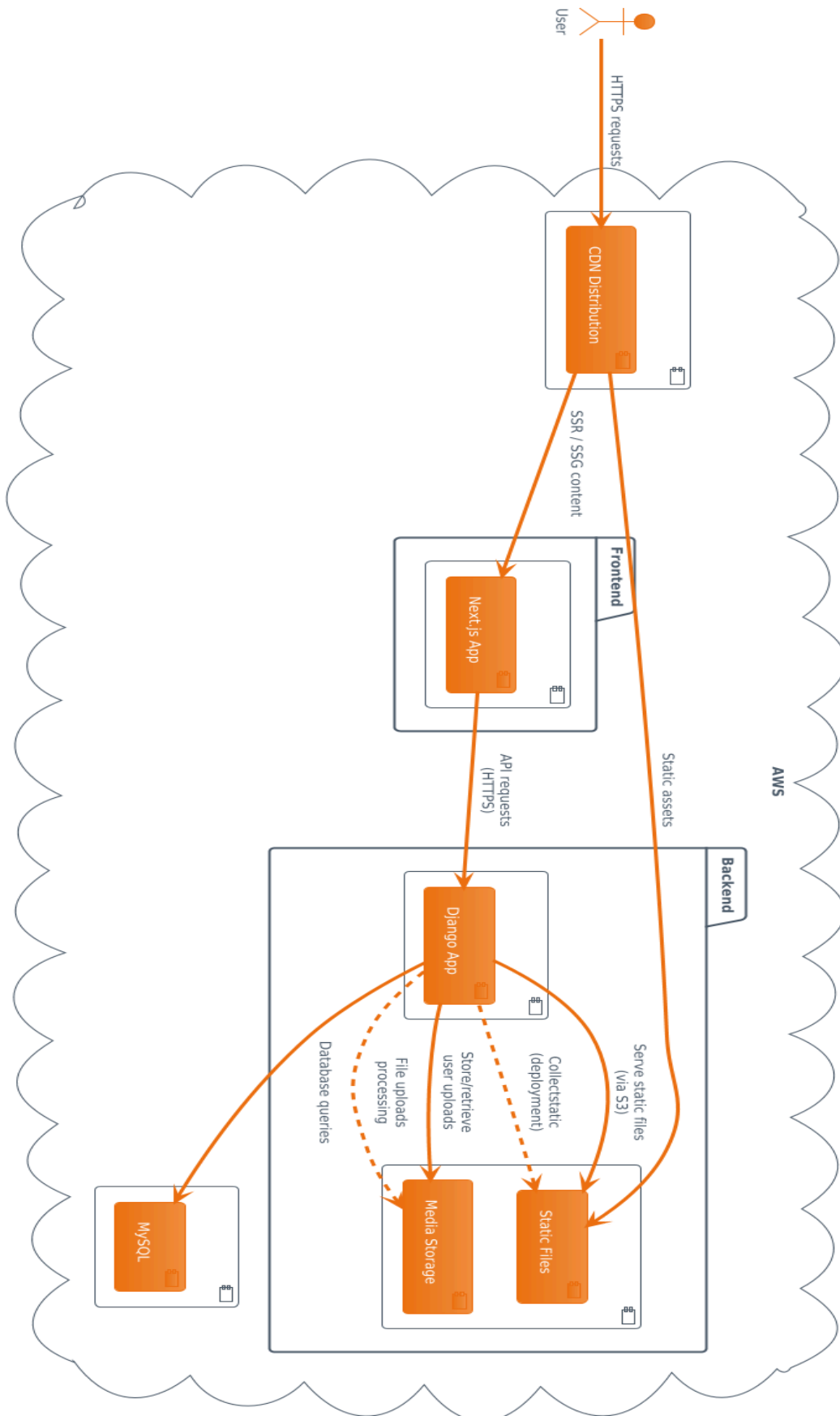
APIs follow REST standards and use JSON format.

5. High Level UML Diagrams

a) UML Class Diagram



b) Component Diagram



6. Identify actual key risks for your project at this time

Skills-Related Risks

1. Limited Expertise in React, Python, and MySQL

Risk

Certain team members lack deep experience with React, Python, or MySQL, which are critical technologies for the project's success.

Potential Impact

This knowledge gap could lead to slower development cycles, lower-quality code, or errors in functionality, ultimately delaying progress and affecting the project's overall performance.

Mitigation Strategy

To address this, we'll implement customized training programs using online resources, tailored to each member's current proficiency. Additionally, we'll foster peer learning by pairing developers with complementary skills, ensuring knowledge transfer between frontend and backend teams to build a more versatile skill set.

2. Insufficient Cloud Deployment Experience

Risk

The team's limited familiarity with deploying applications on cloud platforms like AWS poses a challenge to setting up and maintaining the project's infrastructure.

Potential Impact

Inadequate deployment skills could result in configuration errors, failed deployments, or suboptimal server performance, hindering the application's availability and user experience.

Mitigation Strategy

We'll conduct targeted workshops focused on AWS deployment processes and leverage official documentation for guidance. Practice deployments in a sandbox environment will build confidence, and we'll develop a standardized deployment script to simplify the process for all team members.

Schedule-Related Risks

1. Overly Optimistic Development Timelines

Risk

The team may underestimate the time required for developing and testing features, setting unrealistic expectations for task completion.

Potential Impact

Such miscalculations could lead to missed deadlines, incomplete features, or rushed work, compromising the project's quality and straining team resources.

Mitigation Strategy

To counter this, we'll break down tasks into smaller, well-defined units with clear milestones. By adopting Agile practices, we'll regularly evaluate progress and adjust schedules dynamically to reflect actual development pace and emerging needs.

2. Conflicts with Academic Responsibilities

Risk

Team members' commitments to other academic courses may overlap with project timelines, limiting their availability and focus.

Potential Impact

This could cause delays in task completion, reduced attention to detail, or uneven contributions, potentially jeopardizing the project's quality and schedule.

Mitigation Strategy

We'll start by mapping out each member's academic obligations for the semester and align project tasks to avoid high-pressure periods. Prioritizing key deliverables and setting flexible deadlines will help balance academic and project demands effectively.

Technical Risks

1. Difficulties with Messaging System Integration

Risk

Implementing the in-site messaging system may present unforeseen technical challenges due to its complexity or integration requirements.

Potential Impact

These issues could delay the feature's rollout, introduce bugs, or compromise its reliability, negatively affecting user satisfaction and project timelines.

Mitigation Strategy

We'll allocate extra time for prototyping and rigorous testing to identify issues early. By tapping into online documentation, tutorials, and community forums, we'll troubleshoot problems efficiently and ensure a robust messaging system.

2. Challenges in Managing Digital Media Storage

Risk

Storing and organizing diverse digital media types, such as images, videos, and audio, may lead to inefficiencies if not handled properly.

Potential Impact

Poorly managed storage could slow down access to content, degrade platform performance, or create scalability issues as media volume grows.

Mitigation Strategy

We'll adopt a cloud-based storage solution like Amazon S3, paired with a metadata-driven categorization system for efficient organization. Ongoing monitoring and optimization of storage usage will maintain performance and scalability.

Teamwork-Related Risks

1. Ineffective Team Communication

Risk

Inconsistent or unclear communication among team members, particularly during semester breaks, could disrupt collaboration and coordination.

Potential Impact

Miscommunication might lead to misunderstandings, duplicated efforts, or stalled progress, especially when the team isn't meeting in person.

Mitigation Strategy

We'll establish reliable communication channels, including weekly meetings, WhatsApp for quick updates, and collaborative tools for task tracking. These measures will ensure clarity and alignment, regardless of physical distance.

2. Differing Expectations Among Team Members

Risk

Team members may have varying interpretations of the project's goals, scope, or deliverables, leading to misaligned priorities.

Potential Impact

This could result in wasted effort, conflicting contributions, or deviations from the project's vision, undermining efficiency and cohesion.

Mitigation Strategy

We'll define and document project objectives, roles, and expectations clearly from the outset. Regular check-ins and reviews of milestone documents will keep everyone aligned and focused on shared goals.

Legal/Content-Related Risks

1. Copyright Issues from User-Generated Content

Risk

Users may upload copyrighted material without authorization, exposing the platform to legal risks.

Potential Impact

Such violations could lead to legal action, financial penalties, or damage to the platform's reputation, eroding user trust.

Mitigation Strategy

We'll implement automated content-screening tools to flag potential copyright issues and train administrators on compliance policies. Clear user guidelines on acceptable content will further reduce the risk of violations.

7. Project management

For Milestone 2 and beyond, we aim to improve our workflow by fostering greater independence between the front-end and back-end teams while aligning on shared interfaces. This approach enables parallel development and reduces unnecessary dependencies and delays.

To manage tasks efficiently, we are using Slack for daily communication, with dedicated channels to separate topics and teams—for example, `#frontend`, `#backend`, and `#api-specs`. This helps streamline discussions and keeps team members focused.

For task management, we have adopted **GitHub Projects**, which allows us to create and assign tasks, track progress using boards, and maintain a unified view of all activities. Each task is assigned to a specific team member with clear deadlines. This ensures accountability and transparency.

The combination of GitHub Projects and Slack gives us both granular and strategic control over the project's progress.

By enforcing clear ownership, regular updates, and using structured tools, we ensure that our project remains on track and well-coordinated across all teams.